

# Softmax & Temperature, Worked Out by Hand

When a server asks the host's LLM to complete a sub-task, `modelPreferences.temperature` reshapes a softmax. We compute that distribution at three temperatures, exactly.

**What this document does.** The `sampling/createMessage` request lets an MCP server borrow the host's LLM, and the temperature it sets controls how random the output is. This document derives the softmax, applies temperature, and computes the full probability distribution over a four-token vocabulary at  $T = 1$ ,  $T = 0.5$ , and  $T = 2$  — showing exactly how temperature sharpens or flattens the choice, with the entropy to quantify it.

**Concepts covered:** logits  $\rightarrow$  probabilities · the softmax · temperature scaling  $z/T$  · the  $T \rightarrow 0$  (greedy) and  $T \rightarrow \infty$  (uniform) limits · Shannon entropy · the geometric cost of consent denials.

*Internal learning material. Numbers reproducible from the appendix code. v1.0*

## 1 The setup: four candidate tokens

The LLM produces a *logit* (an unnormalised score) for every vocabulary token. Take a tiny vocabulary of four next-token candidates with logits

$$z = (z_1, z_2, z_3, z_4) = (2.0, 1.0, 0.5, 0.0).$$

“database” scores highest at 2.0; “the” lowest at 0.0. Logits are not probabilities — they can be any real number. Softmax turns them into a distribution.

### Intuition

Logits say *how much* the model prefers each token, on an arbitrary scale. To *sample*, you need a probability distribution: all positive, summing to 1. Softmax is the standard map from “scores” to “probabilities” — it exponentiates (forcing positivity, and amplifying gaps) then normalises.

## 2 Step 1 — The softmax

For logits  $z$ , the softmax probability of token  $i$  is

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Exponentiate each logit, sum, divide. At  $T = 1$ :

$$e^{2.0} = 7.389, \quad e^{1.0} = 2.718, \quad e^{0.5} = 1.649, \quad e^{0.0} = 1.000, \quad \Sigma = 12.756.$$

$$p = \left( \frac{7.389}{12.756}, \frac{2.718}{12.756}, \frac{1.649}{12.756}, \frac{1.000}{12.756} \right) = (0.579, 0.213, 0.129, 0.078).$$

(Check:  $0.579 + 0.213 + 0.129 + 0.078 = 0.999 \approx 1$ . ✓) The top token gets 57.9% — a clear favourite, but the others retain real mass.

### 3 Step 2 — Temperature reshapes the scores

Temperature  $T$  divides the logits *before* softmax:

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

$T < 1$  magnifies the gaps (sharper, more deterministic);  $T > 1$  shrinks them (flatter, more random). Recompute the same  $z$  at  $T = 0.5$  and  $T = 2$ .

$T = 0.5$  (**sharpen**). Scaled logits  $z/T = (4, 2, 1, 0)$ :

$$e^4 = 54.60, e^2 = 7.389, e^1 = 2.718, e^0 = 1, \quad \sum = 65.71,$$

$$p(0.5) = (0.831, 0.112, 0.041, 0.015).$$

$T = 2$  (**flatten**). Scaled logits  $z/T = (1, 0.5, 0.25, 0)$ :

$$e^1 = 2.718, e^{0.5} = 1.649, e^{0.25} = 1.284, e^0 = 1, \quad \sum = 6.651,$$

$$p(2) = (0.409, 0.248, 0.193, 0.150).$$

|                     | $p_1$ | $p_2$ | $p_3$ | $p_4$ | top token |
|---------------------|-------|-------|-------|-------|-----------|
| $T = 0.5$ (sharp)   | 0.831 | 0.112 | 0.041 | 0.015 | 83.1%     |
| $T = 1.0$ (nominal) | 0.579 | 0.213 | 0.129 | 0.078 | 57.9%     |
| $T = 2.0$ (flat)    | 0.409 | 0.248 | 0.193 | 0.150 | 40.9%     |

#### Mini-example — this operation in isolation

Same logit gap, three behaviours. The top-second gap was  $z_1 - z_2 = 1.0$  the whole time; only  $T$  changed. At  $T = 0.5$  that gap becomes 2.0 after scaling and the favourite runs away to 83%; at  $T = 2$  it shrinks to 0.5 and the field bunches up at 41/25/19/15. Temperature never touches the logits' *order* — only how decisively the model commits to it.

### 4 Step 3 — The two limits

$$T \rightarrow 0: \quad p \rightarrow (1, 0, 0, 0) \quad (\text{argmax / greedy}), \quad T \rightarrow \infty: \quad p \rightarrow (\tfrac{1}{4}, \tfrac{1}{4}, \tfrac{1}{4}, \tfrac{1}{4}) \quad (\text{uniform}).$$

As  $T \rightarrow 0$  the largest logit's exponential dominates all others infinitely, so all mass collapses onto the top token — deterministic decoding. As  $T \rightarrow \infty$  every  $z_i/T \rightarrow 0$ , so  $e^{z_i/T} \rightarrow 1$  and all tokens become equally likely — pure noise.

#### Intuition

Temperature is a randomness dial. Near 0 the model always picks its single best guess: repeatable, conservative, good for extraction and classification (the “soft tasks” a server samples for). Near and above 1 it explores alternatives: good for brainstorming, bad for anything that must be exact. A summarisation server should sample cold; a creative server, warm.

## 5 Step 4 — Quantifying randomness: entropy

Shannon entropy measures the uncertainty of the distribution in bits:

$$H(p) = - \sum_i p_i \log_2 p_i, \quad 0 \leq H \leq \log_2 4 = 2 \text{ bits.}$$

At  $T = 1$ :

$$H = -(0.579 \log_2 0.579 + 0.213 \log_2 0.213 + 0.129 \log_2 0.129 + 0.078 \log_2 0.078) = 1.60 \text{ bits.}$$

Sharpening to  $T = 0.5$  drops entropy to  $\approx 0.86$  bits (more certain); flattening to  $T = 2$  raises it to  $\approx 1.90$  bits (almost the 2-bit maximum). Entropy is the single number that says “how unpredictable is the next token.”

## 6 Step 5 — The consent cost of sampling

A sampling request needs user approval. If the user approves with probability  $p$  each time and you retry on denial, the number of attempts until one succeeds is *geometric*, with expectation

$$\mathbb{E}[\text{attempts}] = \frac{1}{p}.$$

At  $p = 0.8$  that is  $1/0.8 = 1.25$  attempts on average; at  $p = 0.5$ , 2 attempts.

### Watch out

This is exactly why a server must handle the deny case gracefully (Module 2.4). Denials are not free retries — each is a round trip and a user interruption. Design sampling requests to be approved on the first ask ( $p \rightarrow 1$ ), not to lean on  $1/p$  retries.

## 7 The argument, at a glance

| # | Idea                    | Formula                                  | Value               |
|---|-------------------------|------------------------------------------|---------------------|
| 1 | Softmax                 | $p_i = e^{z_i} / \sum_j e^{z_j}$         | top = 0.579         |
| 2 | Temperature scaling     | $p_i(T) = e^{z_i/T} / \sum_j e^{z_j/T}$  | —                   |
| 2 | Sharpen $T=0.5$         | top $\uparrow$                           | 0.831               |
| 2 | Flatten $T=2$           | top $\downarrow$                         | 0.409               |
| 3 | Greedy / uniform limits | $T \rightarrow 0 / T \rightarrow \infty$ | (1, 0, 0, 0) / unif |
| 4 | Entropy at $T=1$        | $-\sum p \log_2 p$                       | 1.60 bits           |
| 5 | Approval retries        | $1/p$                                    | 1.25 @ $p=0.8$      |

## 8 Equation cheat-sheet

|                         |                                                                        |
|-------------------------|------------------------------------------------------------------------|
| Softmax                 | $p_i = e^{z_i} / \sum_j e^{z_j}$                                       |
| With temperature        | $p_i(T) = e^{z_i/T} / \sum_j e^{z_j/T}$                                |
| Greedy limit            | $T \rightarrow 0 \Rightarrow p \rightarrow \text{onehot}(\arg \max z)$ |
| Uniform limit           | $T \rightarrow \infty \Rightarrow p_i \rightarrow 1/n$                 |
| Shannon entropy         | $H = -\sum_i p_i \log_2 p_i, 0 \leq H \leq \log_2 n$                   |
| Expected approval tries | $\mathbb{E} = 1/p$ (geometric)                                         |

## A Reference implementation

```
import math

def softmax(z, T=1.0):
    exps = [math.exp(zi / T) for zi in z]
    s = sum(exps)
    return [e / s for e in exps]

def entropy_bits(p):
    return -sum(pi * math.log2(pi) for pi in p if pi > 0)

z = [2.0, 1.0, 0.5, 0.0]
for T in (0.5, 1.0, 2.0):
    p = softmax(z, T)
    print(T, [round(x,3) for x in p], round(entropy_bits(p),2))
# 0.5 [0.831, 0.112, 0.041, 0.015] 0.86
# 1.0 [0.579, 0.213, 0.129, 0.078] 1.60
# 2.0 [0.409, 0.248, 0.193, 0.150] 1.90
```

— end of document —